



Safety Supervisor ECU with Emergency Brake Assist

Submitted By

**Uzair Shaikh
Om Raut
Sahil Deshmukh**

Under The Guidance Of

Isheet Patel

Submitted To

KPIT APEX LAB

Academic Year 2025-2026

ABSTRACT

The Safety Supervisor ECU with Emergency Brake Assist project focuses on the design and development of a prototype Automotive Safety Supervisor Electronic Control Unit (ECU) using the STM32 Nucleo-F446RE microcontroller. The primary objective of the project is to continuously monitor the vehicle's surroundings using sensor data, identify potential collision risks, and automatically activate emergency braking whenever a hazardous situation is detected. The system demonstrates how embedded systems, real-time communication protocols, and intelligent sensor processing can be integrated to improve vehicle safety.

The proposed system uses the STM32 Nucleo-F446RE as the central Safety ECU, which acquires real-time data from a LIDAR-Lite v3HP distance sensor and an MPU6050 Inertial Measurement Unit (IMU). The LIDAR sensor measures the distance between the vehicle and nearby obstacles, while the IMU monitors the vehicle's acceleration and motion. Based on these inputs, the ECU processes the sensor data and classifies the vehicle into three safety states: Normal, Warning, and Emergency. According to the detected state, the system generates visual and audible alerts through LEDs, a buzzer, and an LCD display, and transmits appropriate control commands to the FireBird V Robot.

Communication between the STM32 ECU and the FireBird V Robot is established using the Controller Area Network (CAN) protocol with MCP2515 CAN modules. The FireBird V Robot acts as the experimental vehicle platform and executes movement or emergency braking commands received from the Safety ECU. In addition, a Raspberry Pi 4B is integrated into the system for real-time data logging and dashboard visualization. Sensor readings, safety states, and communication status are recorded and displayed using a Python-based application, making system monitoring and analysis more effective.

The developed prototype successfully demonstrates reliable sensor integration, real-time decision-making, CAN-based communication, automatic emergency braking, and continuous data logging. Experimental testing confirmed that the system responds effectively to different obstacle conditions by transitioning between Normal, Warning, and Emergency states. Although implemented on a robotic platform, the project represents the core concepts of modern Advanced Driver Assistance Systems (ADAS) and provides a strong foundation for future enhancements such as camera-based object detection, GPS integration, cloud-based monitoring, multi-sensor fusion, and intelligent autonomous vehicle safety systems.

Table of Contents

ABSTRACT	i
1. INTRODUCTION	1
2. LITERATURE REVIEW AND RESEARCH PAPERS USED	3
3. HARDWARE AND SOFTWARE TECHNOLOGIES	5
3.1 Introduction	5
3.2 Hardware Used	5
3.2.1 FireBird V Robot	6
3.2.2 STM32 Nucleo-F446RE Development Board	7
3.2.3 Raspberry Pi	8
3.2.4 LIDAR Lite v3HP	9
3.2.5 MPU6050 IMU Sensor	10
3.2.6 MCP2515 CAN Module	11
3.2.7 LCD Display, LEDs, and Buzzer	12
3.3 Software Technologies Used	13
3.3.1 STM32CubeIDE	13
3.3.2 STM32CubeMX	14
3.3.3 Atmel Studio / Arduino IDE	14
3.3.4 Python Programming Language	15
3.3.5 Raspberry Pi OS	15
4. HARDWARE SETUP AND SYSTEM PREPARATION (Week 1)	16
4.1 Introduction	16
4.2 Week Plan	16

4.3 Day-Wise Work Progress	17
4.3.1 Day 1 & Day 2 (19/05/2026 – 20/05/2026)	17
4.3.2 Day 3 (21/05/2026)	18
4.3.3 Day 4 (22/05/2026)	18
4.3.4 Day 5 (23/05/2026)	19
4.4 Work Completed at the End of Week 1	19
5. SENSOR INTERFACING AND DATA ACQUISITION (Week 2)	20
5.1 Introduction	20
5.2 Week Plan	20
5.3 Day-Wise Work Progress	21
5.3.1 Day 6 (25/05/2026)	21
5.3.2 Day 7 (26/05/2026)	22
5.3.3 Day 8 (27/05/2026)	22
5.3.4 Day 9 (28/05/2026)	23
5.4 Work Completed at the End of Week 2	23
6. SAFETY LOGIC DEVELOPMENT (Week 3)	24
6.1 Introduction	24
6.2 Week Plan	24
6.3 Day-Wise Work Progress	25
6.3.1 Day 10 (01/06/2026)	25
6.3.2 Day 11 (02/06/2026)	25
6.3.3 Day 12 (03/06/2026)	26
6.3.4 Day 13 (04/06/2026)	26

6.3.5 Day 14 (05/06/2026)	27
6.4 Work Completed at the End of Week 3	27
7. BRAKE ASSIST SYSTEM IMPLEMENTATION (Week 4)	28
7.1 Introduction	28
7.2 Week Plan	28
7.3 Day-Wise Work Progress	29
7.3.1 Day 15 (08/06/2026)	29
7.3.2 Day 16 (09/06/2026)	29
7.3.3 Day 17 (10/06/2026)	30
7.3.4 Day 18 (11/06/2026)	30
7.3.5 Day 19 (12/06/2026)	31
7.4 Work Completed at the End of Week 4	31
8. COMMUNICATION AND LOGGING (Week 5)	32
8.1 Introduction	32
8.2 Week Plan	32
8.3 Day-Wise Work Progress	33
8.3.1 Day 20 (15/06/2026)	33
8.3.2 Day 21 (16/06/2026)	33
8.3.3 Day 22 (17/06/2026)	34
8.3.4 Day 23 (18/06/2026)	34
8.3.5 Day 24 (19/06/2026)	35
8.4 Work Completed at the End of Week 5	35
9. TESTING AND DOCUMENTATION (Week 6)	36

9.1 Introduction	36
9.2 Week Plan	36
9.3 Day-Wise Work Progress	36
9.3.1 Day 25 (22/06/2026)	36
9.3.2 Day 26 (23/06/2026)	37
9.3.3 Day 27 (24/06/2026)	37
9.3.4 Day 28 & Day 29 (25/06/2026 – 26/06/2026)	37
9.4 Work Completed at the End of Week 6	37
FUTURE IMPROVEMENTS	37
REAL VEHICLE SCENARIO	38
CONCLUSION	40
REFERENCES	41
ACKNOWLEDGEMENT	

1 INTRODUCTION

Modern automobiles are no longer purely mechanical systems. They have evolved into intelligent electronic systems that use multiple Electronic Control Units (ECUs) to monitor vehicle operation and improve passenger safety. Contemporary vehicles contain dozens of ECUs responsible for engine control, braking, steering, transmission, airbag deployment, navigation, and advanced driver assistance features.

One of the major causes of road accidents is delayed driver reaction. Even attentive drivers require a finite amount of time to perceive an obstacle, make a decision, and apply the brakes. During this reaction period, the vehicle continues moving, increasing the likelihood of collision. Advanced Driver Assistance Systems (ADAS) have been developed to reduce these risks by assisting the driver in monitoring the surroundings and taking corrective action whenever necessary. Among these systems, Automatic Emergency Braking (AEB) is one of the most effective technologies for preventing rear-end collisions.

The objective of the Emergency Brake Assist system is to continuously monitor the distance between the vehicle and nearby obstacles. If the driver does not react in time, the system automatically reduces vehicle speed or completely stops the vehicle before impact. The system classifies the vehicle condition into three safety states: Normal (safe distance from obstacle), Warning (obstacle getting closer), and Emergency (collision risk very high). Based on the detected state, appropriate warning indications are generated and braking actions are taken.

This project implements a prototype Safety Supervisor ECU using an STM32 Nucleo-F446RE microcontroller. The ECU performs real-time sensor acquisition from a LIDAR Lite v3HP distance sensor and MPU6050 Inertial Measurement Unit (IMU), processes the data to classify the current safety state, and activates emergency brake control commands when necessary. The FireBird V Robot serves as the experimental vehicle platform, while Raspberry Pi provides visualization and data logging. Communication between different controllers is established using the Controller Area Network (CAN) protocol, which is the standard communication protocol widely used in modern automotive applications.

The project demonstrates how embedded systems, real-time sensor processing, CAN communication, and intelligent safety state classification can be combined to create an effective vehicle safety system. The developed prototype provides a strong foundation for future development of comprehensive automotive safety platforms with additional ADAS features.

2 LITERATURE REVIEW AND RESEARCH PAPERS USED

Before beginning the implementation phase, multiple research papers and technical references related to emergency braking systems, obstacle detection, automotive ECU design, and Advanced Driver Assistance Systems were studied to understand existing technologies, methodologies, limitations, and real-time challenges involved in such systems.

The paper titled "Automatic Emergency Braking System Using Sensor Fusion" discussed how combining multiple sensor inputs such as LIDAR, radar, and camera data can improve the reliability of obstacle detection and emergency braking in automotive applications. The paper highlighted that single-sensor approaches are prone to false detections under varying environmental conditions, and sensor fusion techniques can significantly improve system accuracy and reliability. This paper provided understanding regarding multi-sensor architectures for safety-critical automotive applications.

Another important reference studied was "Design and Implementation of Automotive Electronic Control Units Using STM32 Microcontrollers." This paper focused on the use of STM32 microcontrollers for implementing automotive ECUs and discussed peripheral interfacing, real-time processing, communication protocol integration, and safety state classification techniques. The paper also discussed the importance of CAN communication in automotive embedded systems. This reference was useful for understanding STM32-based ECU implementation approaches and embedded firmware development strategies.

The paper titled "LIDAR-Based Obstacle Detection and Distance Measurement for Autonomous Vehicle Applications" explained the use of LIDAR sensors for real-time obstacle detection in moving vehicle environments. The paper discussed measurement accuracy, sensor calibration methods, and the effect of environmental factors on LIDAR performance. The study highlighted best practices for integrating LIDAR data with embedded controllers for reliable distance estimation. This reference provided understanding regarding LIDAR Lite v3HP sensor characteristics and interfacing techniques relevant to the project.

Another research paper titled "Inertial Measurement Unit Integration for Vehicle Motion Monitoring" explained how IMU sensors such as the MPU6050 can be used to monitor vehicle acceleration, orientation, and motion dynamics. The paper discussed I2C communication protocols, sensor calibration, and data filtering methods for improving measurement accuracy. This study helped in understanding MPU6050 interfacing with STM32 microcontrollers and the role of IMU data in vehicle safety state classification.

The paper titled "Controller Area Network (CAN) Protocol Implementation for Automotive ECU Communication" explained the CAN communication protocol used in modern automotive embedded systems. The paper discussed CAN frame structures, arbitration mechanisms, error handling, and implementation using MCP2515 CAN controllers. The study provided detailed

understanding regarding CAN bus integration between multiple embedded controllers and was directly applied during the CAN communication setup between the STM32 ECU and the FireBird V Robot in the project.

The literature review process provided detailed understanding regarding obstacle detection algorithms, automotive ECU design, sensor fusion, CAN communication, real-time embedded processing, and Advanced Driver Assistance Systems. The knowledge gained from these studies was directly applied during hardware selection, firmware development, safety logic implementation, and system integration phases of the project.

3 HARDWARE AND SOFTWARE

3.1 Introduction

The project "Safety Supervisor ECU with Emergency Brake Assist" uses a combination of embedded hardware platforms, sensor modules, communication interfaces, and driver alert modules to create a real-time intelligent vehicle safety system. The overall architecture is designed as a prototype to demonstrate the feasibility of using low-cost embedded hardware for implementing automotive safety supervisor functionalities.

3.2 Hardware Used

The hardware architecture of the project consists of embedded processing units, sensing devices, communication interfaces, and driver alert systems integrated together to create a real-time intelligent vehicle safety platform. The hardware components are selected to support obstacle detection, safety state classification, CAN communication, and emergency brake control operations.

3.2.1 FireBird V Robot

The FireBird V Robot is used as the experimental vehicle platform of the system. It is built on an ATmega2560 microcontroller combined with a Raspberry Pi 4B for higher-level processing. The robot provides motor control, movement capabilities, and serves as the target platform for receiving and executing braking commands from the STM32 Safety Supervisor ECU.

The FireBird V acts as the vehicle in this prototype setup. It receives control commands from the STM32 ECU through CAN communication and adjusts motor speed or activates emergency braking based on the commands received. The robot platform was selected because of its dual-controller architecture, CAN interface support, and suitability for embedded automotive safety system demonstrations.

3.2.2 STM32 Nucleo-F446RE Development Board

The STM32 Nucleo-F446RE development board is used as the primary Safety Supervisor ECU of the system. The board is based on the STM32F446RE microcontroller from STMicroelectronics and supports high-speed embedded processing, multiple communication peripherals, timer operations, I2C interfacing, SPI interfacing, and GPIO control.

The STM32 board is responsible for reading sensor data from LIDAR and IMU sensors, processing the data to classify the current safety state, generating driver alert outputs, and transmitting emergency braking commands to the FireBird V Robot through CAN communication. It serves as the central intelligence unit of the Safety Supervisor ECU.

The board was selected because of its reliability in embedded applications, multiple communication peripherals including SPI and I2C, real-time processing capability, and compatibility with automotive embedded development tools.

3.2.3 Raspberry Pi

Raspberry Pi is used for data logging, system visualization, and higher-level monitoring in the project. It receives system state information from the STM32 ECU and logs the data for analysis and dashboard visualization purposes.

Raspberry Pi provides a Linux-based computing environment capable of running Python scripts for data logging, serial communication handling, and visualization applications. It complements the STM32 ECU by providing additional processing and logging capabilities that are not feasible on the microcontroller alone.

3.2.4 LIDAR Lite v3HP

The LIDAR Lite v3HP is used as the primary obstacle distance measurement sensor in the system. The sensor uses laser-based distance measurement and provides accurate range readings that are processed by the STM32 ECU for collision risk assessment.

The LIDAR sensor continuously measures the distance between the vehicle and obstacles in its field of view. The STM32 ECU reads the LIDAR distance data through the I2C interface and uses it as the primary input for safety state classification and emergency braking decisions.

3.2.5 MPU6050 IMU Sensor

The MPU6050 Inertial Measurement Unit is used to monitor vehicle motion, acceleration, and orientation in real time. The sensor combines a 3-axis accelerometer and a 3-axis gyroscope in a single integrated circuit and communicates with the STM32 ECU through the I2C interface.

IMU data provides additional context about vehicle dynamics during safety state classification. Sudden acceleration or deceleration data from the MPU6050 complements LIDAR distance measurements for more reliable collision risk assessment.

3.2.6 MCP2515 CAN Module

The MCP2515 is a standalone CAN controller with an integrated SPI interface, used to implement CAN communication between the STM32 ECU and the FireBird V Robot. Two MCP2515 modules are used — one connected to the STM32 Nucleo and one interfaced with the FireBird V platform — to establish a CAN bus network.

CAN communication provides a reliable, noise-resistant, multi-node communication channel that is the standard protocol used in automotive embedded networks. The MCP2515 modules handle CAN frame transmission, reception, and error management independently from the main microcontrollers.

3.2.7 LCD Display, LEDs, and Buzzer

An LCD display is used to show real-time system state information, sensor readings, and warning messages to the operator. LEDs provide visual warning indications corresponding to each safety state (Normal, Warning, Emergency), while a buzzer provides audible alerts during warning and emergency conditions.

These alert mechanisms improve operator awareness and simulate the driver notification systems used in modern intelligent vehicles.

3.3 Software Technologies Used

The software section includes embedded development environments, communication tools, programming languages, and visualization platforms used for system development and testing.

3.3.1 STM32CubeIDE

STM32CubeIDE is used as the primary embedded firmware development environment for the STM32 Nucleo-F446RE. It provides an integrated development environment with code editing, compilation, debugging, and program flashing support for STM32 microcontrollers.

3.3.2 STM32CubeMX

STM32CubeMX is used for graphical configuration of STM32 peripherals including I2C, SPI, CAN, GPIO, and timer modules. The tool automatically generates peripheral initialization code, reducing firmware development time.

3.3.3 Atmel Studio / Arduino IDE

Atmel Studio is used for programming and debugging the ATmega2560 microcontroller on the FireBird V Robot platform. Arduino IDE was also used during communication testing and debugging activities due to its simpler serial debugging environment.

3.3.4 Python Programming Language

Python is used on Raspberry Pi for data logging, serial communication handling, and visualization scripting. Python provides extensive library support for serial communication, data processing, and dashboard integration.

3.3.5 Raspberry Pi OS

Raspberry Pi OS provides the Linux-based operating system environment on the Raspberry Pi platform. It supports Python development, serial communication tools, and network-based data transfer for logging and visualization.

4 HARDWARE SETUP AND SYSTEM PREPARATION (Week 1)

4.1 Introduction

The first week of the project focused on hardware setup, software installation, system environment preparation, and initial connectivity verification. Activities included setting up the FireBird V Robot, configuring the STM32 Nucleo-F446RE development board, installing required software tools, and performing initial hardware verification tests. Research activities related to automotive ECU design, emergency braking systems, and CAN communication were also carried out during this phase.

Multiple hardware and software challenges were encountered during initial system setup, particularly during robot connectivity testing, STM32 peripheral configuration, and software installation. Various debugging and verification methods were applied to resolve issues and prepare the hardware and software foundation for the subsequent development phases.

4.2 Week Plan

The planned activities for Week 1 were:

1. Set up and verify the FireBird V Robot hardware
2. Set up and configure the STM32 Nucleo-F446RE development board
3. Install STM32CubeIDE, STM32CubeMX, and Atmel Studio
4. Install Raspberry Pi OS and configure the Raspberry Pi environment
5. Test basic robot movement and motor control
6. Verify LIDAR and IMU sensor availability and connectivity
7. Plan CAN communication architecture between STM32 and FireBird V
8. Begin project research and literature review activities

4.3 Day-Wise Work Progress

4.3.1 Day 1 & Day 2 (19/05/2026 – 20/05/2026)

4.3.1.1 Work Done

The initial two days were dedicated to project research, system planning, and understanding hardware requirements. Existing automotive ECU designs, emergency braking systems, CAN communication architectures, and ADAS technologies were studied in detail. Hardware datasheets for STM32 Nucleo-F446RE, FireBird V Robot, LIDAR Lite v3HP, MPU6050, and MCP2515 were reviewed to understand interfacing and compatibility requirements.

The first project planning meeting was conducted to finalize the overall project architecture, communication strategy, and implementation phases. The six-phase development plan was

discussed and confirmed. Required software tools were identified and installation was initiated on the development workstations.

4.3.1.2 Problems Faced

9. Large volume of research material and multiple implementation approaches created initial planning confusion.
10. Some required software packages had version dependency conflicts during installation.

4.3.1.3 Solutions Implemented

11. A simplified phase-wise implementation plan was created to organize the project activities systematically.
12. Software versions were verified against hardware compatibility requirements before installation.

4.3.2 Day 3 (21/05/2026)

4.3.2.1 Work Done

STM32CubeIDE and STM32CubeMX were installed and configured successfully on the development workstation. Basic LED blinking programs were compiled and uploaded to the STM32 Nucleo-F446RE to verify proper hardware operation, ST-LINK connectivity, and debug interface functionality.

Raspberry Pi OS was flashed onto a microSD card using Raspberry Pi Imager and initial boot configuration was performed. Atmel Studio was also installed for ATmega2560 firmware development on the FireBird V Robot platform.

4.3.2.2 Problems Faced

13. ST-LINK debugger was not detected on the first attempt due to driver installation issues.
14. Raspberry Pi OS flashing failed initially due to microSD card reader compatibility issues.

4.3.2.3 Solutions Implemented

15. ST-LINK USB drivers were manually reinstalled to resolve detection issues.
16. A different microSD card reader was used successfully for OS flashing.

4.3.3 Day 4 (22/05/2026)

4.3.3.1 Work Done

The FireBird V Robot was powered on and initial connectivity tests were performed. Serial communication between the FireBird V and the development workstation was established for

testing and debugging. Basic motor movement commands were tested to verify robot motor control functionality.

STM32 GPIO configuration was performed using STM32CubeMX for LED, buzzer, and future sensor interfacing. Timer configurations for PWM output were also initiated during this phase.

4.3.3.2 Problems Faced

17. FireBird V Robot serial port was not recognized on the development workstation.
18. Motor control test commands did not produce expected movement results initially.

4.3.3.3 Solutions Implemented

19. Correct COM port settings and baud rate configuration were applied for serial communication.
20. Motor control registers were reverified using FireBird V documentation and example programs.

4.3.4 Day 5 (23/05/2026)

4.3.4.1 Work Done

FireBird V Robot basic motor movement was verified successfully. Forward, backward, and stop commands were tested and confirmed to be working correctly. Serial debugging output was also verified for proper data transmission between the robot and workstation.

Physical hardware layout planning was conducted for the complete system assembly including STM32 Nucleo, LIDAR sensor, IMU sensor, CAN modules, LCD display, LEDs, and buzzer connections. Initial wiring diagrams were prepared for future hardware integration.

4.3.4.2 Problems Faced

21. Some GPIO pin assignments conflicted with existing FireBird V hardware connections.
22. I2C bus address conflicts were anticipated for future LIDAR and IMU sensor integration.

4.3.4.3 Solutions Implemented

23. GPIO pin assignments were reorganized to avoid conflicts with existing robot hardware.
24. I2C addresses for LIDAR Lite v3HP and MPU6050 were verified from datasheets to confirm they are different and can coexist on the same I2C bus.

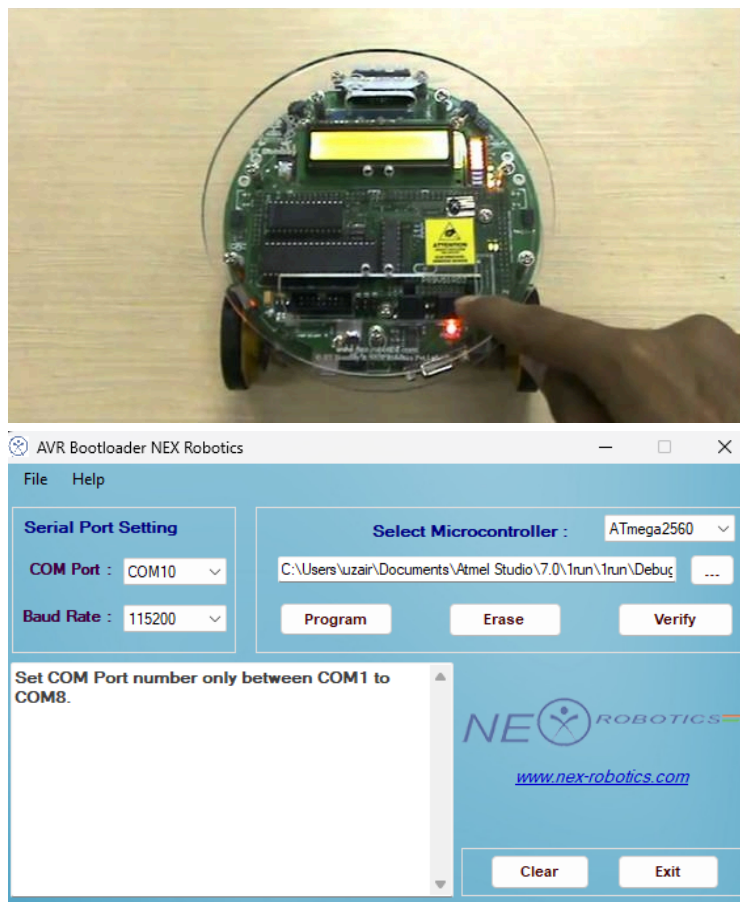
4.4 Work Completed at the End of Week 1

By the end of Week 1, the following tasks were completed successfully:

25. STM32 Nucleo-F446RE setup and basic program verification completed

26. Raspberry Pi OS installation and configuration completed
27. Atmel Studio and STM32CubeIDE installation completed
28. FireBird V Robot basic movement verification completed
29. Serial communication between workstation and FireBird V confirmed
30. Hardware layout planning and wiring diagram preparation initiated
31. Project research and literature review completed
32. System architecture and phase-wise implementation plan finalized

The first week established the hardware and software foundation required for sensor interfacing, safety logic development, CAN communication, and emergency brake implementation in subsequent project phases.



5 SENSOR INTERFACING AND DATA ACQUISITION (Week 2)

5.1 Introduction

The second week of the project focused on interfacing the LIDAR Lite v3HP distance sensor and MPU6050 IMU sensor with the STM32 Nucleo-F446RE development board. During this phase, I2C communication protocols were configured and tested for both sensors, and initial sensor data reading and validation were performed. Sensor data quality, measurement accuracy, and real-time acquisition speed were evaluated for suitability in safety-critical collision detection applications.

Multiple hardware and software challenges were encountered during sensor interfacing, particularly related to I2C communication configuration, sensor initialization sequences, and data validation. Various debugging methods including logic analyzer testing, serial output verification, and register-level debugging were used to identify and resolve issues.

5.2 Week Plan

The planned activities for Week 2 were:

33. Connect LIDAR Lite v3HP to STM32 via I2C interface
34. Configure I2C peripheral on STM32 using STM32CubeMX
35. Read and validate LIDAR distance measurements
36. Connect MPU6050 IMU sensor to STM32 via I2C interface
37. Read accelerometer and gyroscope data from MPU6050
38. Implement data filtering for sensor noise reduction
39. Display sensor readings on LCD for verification
40. Begin CAN module hardware setup

5.3 Day-Wise Work Progress

5.3.1 Day 6 (25/05/2026)

5.3.1.1 Work Done

I2C peripheral configuration for the STM32 Nucleo-F446RE was performed using STM32CubeMX. The I2C1 interface was configured at 400 kHz (Fast Mode) for communication with the LIDAR Lite v3HP sensor. Initial register-level communication with the LIDAR sensor was tested and the sensor responded to initialization commands correctly.

Basic distance reading functionality was implemented and initial LIDAR measurement values were displayed on the serial monitor for verification. The sensor successfully returned distance values within expected ranges during static testing.

5.3.1.2 Problems Faced

41. Initial I2C communication returned incorrect acknowledgment signals.
42. LIDAR sensor required specific initialization sequences that were not immediately clear from basic documentation.

5.3.1.3 Solutions Implemented

43. I2C bus pull-up resistor values and voltage levels were verified and corrected.
44. LIDAR Lite v3HP application note and register map documentation were studied to implement correct initialization and measurement sequences.

5.3.2 Day 7 (26/05/2026)

5.3.2.1 Work Done

LIDAR distance measurement was refined and continuous real-time reading was implemented with a stable polling interval. Distance values were verified by placing objects at known distances and comparing measured results with actual measurements.

MPU6050 IMU sensor interfacing was initiated. The sensor was connected to the STM32 I2C bus and initialization commands were sent to configure the accelerometer and gyroscope sensitivity ranges. Initial raw data readings from both accelerometer and gyroscope registers were obtained and displayed on the serial monitor.

5.3.2.2 Problems Faced

45. LIDAR measurements showed occasional incorrect readings during rapid object movement.
46. MPU6050 I2C address configuration required verification to avoid bus conflicts with LIDAR sensor.

5.3.2.3 Solutions Implemented

47. Multiple-sample averaging was implemented to reduce LIDAR measurement noise during dynamic testing.
48. MPU6050 I2C address was confirmed as 0x68 while LIDAR Lite v3HP operates at 0x62, confirming no address conflicts on the shared I2C bus.

5.3.3 Day 8 (27/05/2026)

5.3.3.1 Work Done

MPU6050 accelerometer and gyroscope data reading was refined. Sensitivity settings were configured for appropriate ranges suitable for vehicle motion monitoring. A basic complementary filter was implemented to combine accelerometer and gyroscope readings for improved orientation estimation.

LCD interfacing with STM32 was initiated to display real-time sensor readings including LIDAR distance values and IMU acceleration data. Display initialization and data formatting routines were implemented.

5.3.3.2 Problems Faced

- 49. LCD display showed scrambled characters during initial initialization.
- 50. IMU data showed significant drift in gyroscope readings over time without calibration.

5.3.3.3 Solutions Implemented

- 51. LCD initialization timing and data write sequences were corrected based on datasheet specifications.
- 52. Gyroscope bias calibration routine was implemented at system startup to reduce drift.

5.3.4 Day 9 (28/05/2026)

5.3.4.1 Work Done

Complete dual-sensor data acquisition was tested with both LIDAR and MPU6050 operating simultaneously on the shared I2C bus. Real-time sensor data was displayed on the LCD and also transmitted through UART serial interface for monitoring on the development workstation.

MCP2515 CAN module hardware setup was initiated. SPI communication between STM32 and MCP2515 was configured using STM32CubeMX, and initial register access tests were performed to verify SPI connectivity.

5.3.4.2 Problems Faced

- 53. Simultaneous I2C transactions to LIDAR and MPU6050 occasionally caused bus timing conflicts.
- 54. MCP2515 SPI communication returned incorrect responses during initial testing.

5.3.4.3 Solutions Implemented

- 55. Sequential I2C polling with appropriate delays between LIDAR and IMU reads was implemented to prevent bus conflicts.
- 56. SPI clock polarity and phase settings for MCP2515 were corrected after reviewing the MCP2515 datasheet communication requirements.

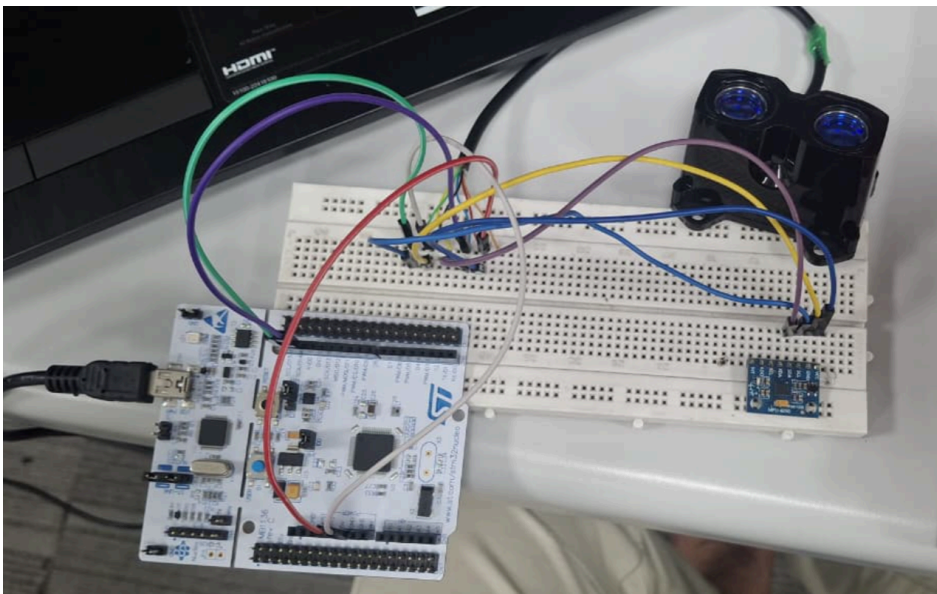
5.4 Work Completed at the End of Week 2

By the end of Week 2, the following tasks were completed successfully:

- 57. LIDAR Lite v3HP sensor interfacing with STM32 via I2C completed

58. Real-time distance measurement and validation completed
59. MPU6050 IMU sensor interfacing with STM32 via I2C completed
60. Accelerometer and gyroscope data reading implemented
61. Complementary filter for IMU data implemented
62. LCD display interfacing and sensor data visualization completed
63. Dual-sensor simultaneous operation tested and verified
64. MCP2515 SPI communication setup initiated

Week 2 successfully established the sensor data acquisition foundation of the project by implementing reliable real-time readings from both the LIDAR distance sensor and MPU6050 IMU sensor, providing the input data required for safety state classification in the subsequent development phase.



```
DISTANCE: 206 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 202 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 198 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 1 cm   SPEED: 0.15 m/s  STATE: EMERGENCY
DISTANCE: 0 cm   SPEED: 0.15 m/s  STATE: EMERGENCY
DISTANCE: 1 cm   SPEED: 0.15 m/s  STATE: EMERGENCY
DISTANCE: 193 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 207 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 213 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 215 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 218 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 217 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 218 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 217 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 217 cm  SPEED: 0.15 m/s  STATE: NORMAL
DISTANCE: 215 cm  SPEED: 0.15 m/s  STATE: NORMAL
```

6 SAFETY LOGIC DEVELOPMENT (Week 3)

6.1 Introduction

The third week of the project focused on implementing the core safety logic of the Safety Supervisor ECU. During this phase, sensor data from the LIDAR and IMU was processed by the STM32 to classify the current vehicle safety state into three categories: Normal, Warning, and Emergency. Alert outputs including LEDs and buzzer were integrated and tested to verify that the correct notifications were generated for each safety state. State transition logic was also refined and validated through systematic obstacle movement testing.

6.2 Week Plan

The planned activities for Week 3 were:

65. Process LIDAR distance data for safety state classification
66. Implement three-level safety state machine: Normal, Warning, Emergency
67. Configure threshold distances for state transitions
68. Integrate LED visual indicators for each safety state
69. Integrate buzzer audible alerts for Warning and Emergency states
70. Test state switching using obstacle movement
71. Continue CAN communication module setup and testing
72. Verify safety logic reliability under varying test conditions

6.3 Day-Wise Work Progress

6.3.1 Day 10 (01/06/2026)

6.3.1.1 Work Done

Safety state classification logic was designed and implemented in STM32 firmware. Distance thresholds were defined for transitioning between Normal, Warning, and Emergency states based on LIDAR distance measurements. The state machine logic was coded and integrated with the sensor reading functions.

LED outputs were configured on STM32 GPIO pins to provide visual state indication: green LED for Normal state, yellow LED for Warning state, and red LED for Emergency state. Initial state machine testing was performed using fixed-distance objects.

6.3.1.2 Problems Faced

73. State transitions were occurring too rapidly due to noise in LIDAR readings near threshold distances.
74. LED GPIO configurations conflicted with I2C peripheral pin assignments on the STM32.

6.3.1.3 Solutions Implemented

- 75. Hysteresis logic was added to the state machine to prevent rapid state oscillations near threshold boundaries.
- 76. GPIO pin assignments were reorganized to resolve conflicts with I2C and other peripheral pins.

6.3.2 Day 11 (02/06/2026)

6.3.2.1 Work Done

Buzzer integration was completed. The buzzer was configured to produce different alert patterns for Warning and Emergency states, providing clear audio differentiation between the two alert levels. Buzzer timing was implemented using STM32 timer interrupts to avoid blocking the main sensor reading loop.

Safety state thresholds were refined through testing. Multiple obstacle distance scenarios were tested to validate the correct state transitions and verify that the system responded appropriately under different conditions.

6.3.2.2 Problems Faced

- 77. Buzzer alert timing interfered with the sensor reading cycle when implemented in the main loop.
- 78. Some obstacle positions produced inconsistent state readings near transition boundaries.

6.3.2.3 Solutions Implemented

- 79. Buzzer control was moved to a timer interrupt service routine to prevent interference with the main sensor acquisition loop.
- 80. Moving average filter was applied to LIDAR readings before state classification to improve stability near transition boundaries.

6.3.3 Day 12 (03/06/2026)

6.3.3.1 Work Done

IMU acceleration data was integrated into the safety logic. Sudden deceleration events detected by the MPU6050 were used as an additional input to the safety state machine, providing supplementary information about vehicle dynamics during collision scenarios.

LCD display was updated to show the current safety state name, LIDAR distance value, and IMU acceleration data in real time. This provided a comprehensive visual interface for monitoring system status during testing.

6.3.3.2 Problems Faced

- 81. IMU integration required careful tuning of acceleration thresholds to avoid false emergency triggers from normal vehicle vibrations.
- 82. LCD update frequency caused minor delays in the sensor reading cycle.

6.3.3.3 Solutions Implemented

- 83. IMU thresholds were conservatively set based on empirical testing to differentiate between normal vibrations and genuine sudden deceleration events.
- 84. LCD update rate was reduced and display writes were scheduled less frequently to minimize impact on sensor acquisition timing.

6.3.4 Day 13 (04/06/2026)

6.3.4.1 Work Done

Complete safety state machine testing was performed using dynamic obstacle movement scenarios. Objects were moved at different speeds toward the sensor to verify that state transitions occurred at correct distances and with appropriate timing. Alert outputs were verified to activate and deactivate correctly during state transitions.

CAN module testing continued in parallel. MCP2515 initialization was completed and basic CAN frame transmission was verified between two MCP2515 modules using a loopback test configuration.

6.3.4.2 Problems Faced

- 85. CAN loopback test initially failed due to incorrect bit timing configuration for the MCP2515.
- 86. State machine response timing was slower than expected during rapid obstacle approach scenarios.

6.3.4.3 Solutions Implemented

- 87. CAN bit timing registers were recalculated based on the 8 MHz crystal frequency of the MCP2515 and corrected configuration values were applied.
- 88. Sensor reading and state update loop frequency was increased to improve response time during rapid obstacle approach scenarios.

6.3.5 Day 14 (05/06/2026)

6.3.5.1 Work Done

End-to-end safety logic validation was completed. The complete system from sensor reading to state classification to alert output was tested continuously over an extended period to verify system stability and reliability. Response time measurements were taken to quantify system latency from obstacle detection to alert activation. Safety state table was finalized and

documented. CAN communication testing continued with initial attempts to transmit state information between STM32 and FireBird V platforms.

6.3.5.2 Problems Faced

89. Long-duration testing revealed occasional sensor timeout errors during I2C communication.
90. CAN communication between STM32 and FireBird V did not succeed during initial integration tests.

6.3.5.3 Solutions Implemented

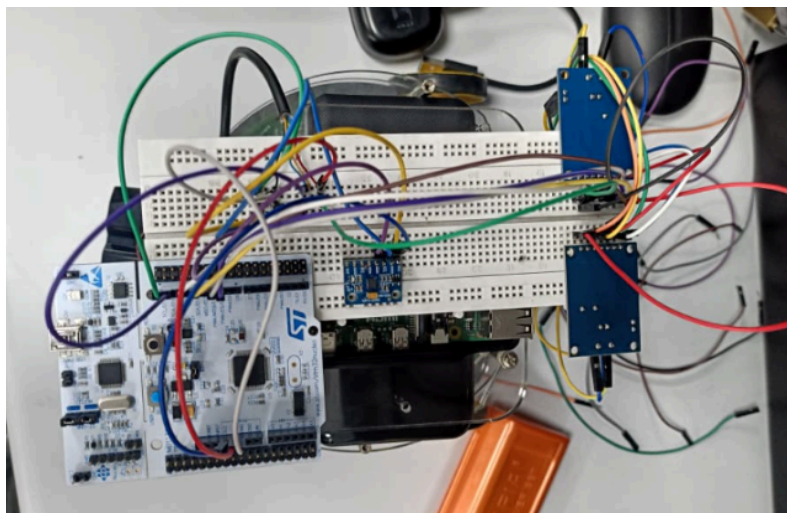
91. I2C timeout recovery routines were implemented to reset the I2C peripheral and retry communication after timeout errors.
92. CAN communication debugging was planned as a focused activity for Week 5 after completing brake assist system integration.

6.4 Work Completed at the End of Week 3

By the end of Week 3, the following tasks were completed successfully:

93. Safety state machine with Normal, Warning, and Emergency states implemented
94. LIDAR-based safety state classification logic completed
95. IMU-based supplementary state input integrated
96. LED visual indicators for each safety state configured and tested
97. Buzzer audible alert system with timer-based control implemented
98. LCD real-time display of safety state, distance, and IMU data completed
99. Hysteresis and moving average filtering for state stability implemented
100. Safety state machine validated through dynamic obstacle testing
101. MCP2515 CAN module initialization and loopback testing completed

Week 3 successfully established the core intelligence of the Safety Supervisor ECU by implementing a complete, reliable safety state classification system with multi-level alert outputs, providing the foundation for brake control command transmission in the subsequent project phases.



7 BRAKE ASSIST SYSTEM IMPLEMENTATION (Week 4)

7.1 Introduction

The fourth week of the project focused on implementing the Emergency Brake Assist control system. During this phase, control command generation based on safety state classification was developed, and initial integration between the STM32 Safety Supervisor ECU and the FireBird V Robot motor control system was attempted. PWM-based motor speed reduction logic was designed and motor stop commands for Emergency braking were implemented in the FireBird V firmware.

7.2 Week Plan

The planned activities for Week 4 were:

102. Design brake assist command structure for FireBird V Robot
103. Implement motor speed reduction logic for Warning state
104. Implement complete motor stop logic for Emergency state
105. Test motor control responses with simulated state commands
106. Initiate STM32 to FireBird V brake command transmission
107. Verify correct motor behavior in response to safety state changes
108. Continue CAN communication integration between ECU and robot
109. Test end-to-end brake assist functionality

7.3 Day-Wise Work Progress

7.3.1 Day 15 (08/06/2026)

7.3.1.1 Work Done

Brake assist command structure was designed. Three command types were defined: Normal (continue at current speed), Warning (reduce speed to 50% of current speed), and Emergency (complete motor stop). Command encoding format was designed for reliable transmission over CAN communication.

FireBird V Robot firmware was modified to include a brake command reception handler that reads incoming commands and adjusts motor control accordingly. Basic motor speed reduction and stop functions were implemented and tested locally on the FireBird V platform without CAN communication.

7.3.1.2 Problems Faced

110. FireBird V motor control registers required specific timing sequences for smooth speed reduction.
111. Immediate motor stop from full speed caused mechanical stress on the robot platform.

7.3.1.3 Solutions Implemented

- 112. Gradual motor speed reduction was implemented for Warning state to ensure smooth deceleration.
- 113. Emergency stop was implemented as a rapid but controlled deceleration rather than an instantaneous hard stop to protect robot hardware during testing.

7.3.2 Day 16 (09/06/2026)

7.3.2.1 Work Done

Motor speed reduction logic for the Warning state was tested systematically. The robot was operated at various speeds and Warning state commands were sent to verify that speed reduction occurred correctly and proportionally. Motor stop functionality for Emergency state was also tested from different initial speed conditions.

CAN transmission code was developed on the STM32 side to encode safety state information into CAN frames and transmit them using the MCP2515 module.

7.3.2.2 Problems Faced

- 114. Speed reduction response was inconsistent at very low initial motor speeds.
- 115. CAN frame transmission from STM32 succeeded but FireBird V CAN reception was not verified yet.

7.3.2.3 Solutions Implemented

- 116. Minimum speed threshold was defined below which Warning state commands would directly trigger Emergency stop instead of attempting further speed reduction.
- 117. CAN reception verification on FireBird V was planned as a priority for the following day.

7.3.3 Day 17 (10/06/2026)

7.3.3.1 Work Done

CAN reception on the FireBird V Robot platform was implemented and tested. MCP2515 interrupt-based CAN frame reception was configured on the FireBird side and initial received frame data was displayed on the serial monitor for verification.

End-to-end transmission of brake commands from STM32 to FireBird V through CAN was tested. Safety state commands were manually triggered on the STM32 and the corresponding motor behavior changes were observed on the FireBird V Robot.

7.3.3.2 Problems Faced

- 118. CAN reception interrupt on FireBird V was not triggering correctly due to interrupt priority configuration issues.
- 119. Received CAN frame data occasionally showed bit errors during initial testing.

7.3.3.3 Solutions Implemented

- 120. Interrupt priority configuration for CAN reception on the ATmega2560 was corrected and reception was verified successfully.
- 121. CAN termination resistors were added to both ends of the CAN bus to improve signal integrity and reduce bit errors.

7.3.4 Day 18 (11/06/2026)

7.3.4.1 Work Done

Complete end-to-end brake assist system testing was performed. The FireBird V Robot was set to move forward and obstacles were placed at varying distances from the LIDAR sensor. Safety state transitions were verified to trigger correct brake commands, and motor behavior was observed to confirm appropriate speed reduction and emergency stop responses.

Warning state correctly triggered gradual speed reduction. Emergency state correctly triggered motor stop. Normal state correctly allowed full speed operation. The complete brake assist logic was verified to work as designed.

7.3.4.2 Problems Faced

- 122. CAN communication latency occasionally caused delayed brake command arrival at the FireBird V.
- 123. Robot required some distance to come to a complete stop after Emergency command was received due to mechanical inertia.

7.3.4.3 Solutions Implemented

- 124. CAN message priority was configured to give brake commands the highest priority for fastest transmission.
- 125. Emergency state threshold distance was adjusted to account for robot stopping distance, ensuring the Emergency command was sent early enough for safe stopping.

7.3.5 Day 19 (12/06/2026)

7.3.5.1 Work Done

Brake assist system performance was evaluated over repeated test runs. Stopping distance measurements were taken for different initial robot speeds and compared against threshold distance settings. System reliability and response consistency were assessed across multiple test cycles.

Firmware documentation for brake assist logic was prepared. Initial plans for Week 5 CAN logging and Raspberry Pi integration were outlined.

7.3.5.2 Problems Faced

126. Stopping distance varied slightly between test runs due to minor differences in robot speed and floor surface friction.

7.3.5.3 Solutions Implemented

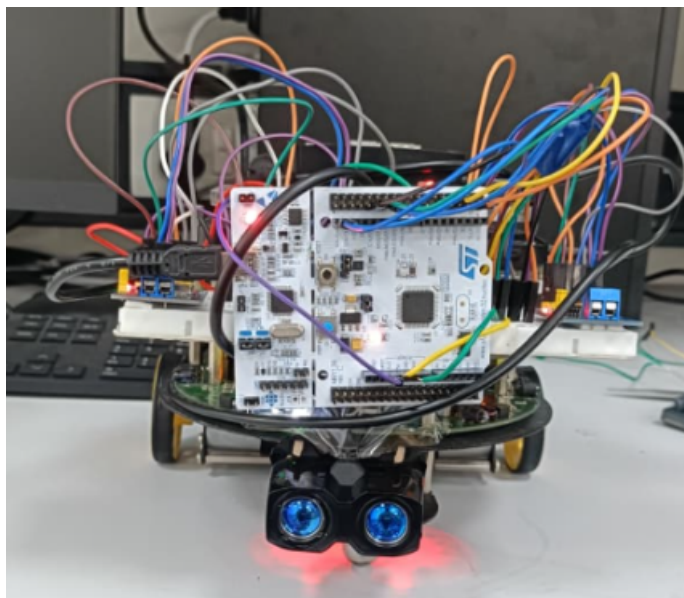
127. A safety margin buffer was added to the Emergency state threshold distance to account for stopping distance variability.

7.4 Work Completed at the End of Week 4

By the end of Week 4, the following tasks were completed successfully:

128. Brake assist command structure designed and implemented
129. Warning state motor speed reduction logic implemented
130. Emergency state motor stop logic implemented
131. CAN transmission of brake commands from STM32 completed
132. CAN reception of brake commands on FireBird V completed
133. End-to-end brake assist system functionality verified
134. Stopping distance measurement and threshold adjustment completed
135. CAN bus termination and signal integrity improvement completed

Week 4 successfully established the emergency brake assist control foundation of the project, demonstrating end-to-end safety command transmission from the STM32 Safety Supervisor ECU to the FireBird V Robot through CAN communication with correct motor control responses.



8 COMMUNICATION AND LOGGING (Week 5)

8.1 Introduction

The fifth week of the project focused on establishing comprehensive data logging, system monitoring, and visualization capabilities. During this phase, Raspberry Pi was integrated into the system for data logging and real-time monitoring. UART-based communication between the STM32 ECU and Raspberry Pi was established for log data transmission. Python-based logging scripts were developed and a dashboard was created for visualizing system state, distance data, and alert events.

8.2 Week Plan

The planned activities for Week 5 were:

136. Configure UART communication between STM32 and Raspberry Pi
137. Develop Python logging script on Raspberry Pi
138. Log safety state, LIDAR distance, IMU data, and alert events
139. Implement LCD display updates for real-time system monitoring
140. Develop dashboard visualization for logged data
141. Test complete logging and visualization pipeline
142. Verify CAN and UART communication stability during extended operation
143. Prepare system for final integration testing

8.3 Day-Wise Work Progress

8.3.1 Day 20 (15/06/2026)

8.3.1.1 Work Done

UART communication between STM32 Nucleo-F446RE and Raspberry Pi was configured. STM32 UART peripheral was set up at 115200 baud rate and a structured data packet format was designed for transmitting safety state, distance readings, IMU data, and timestamp information to the Raspberry Pi.

Initial UART transmission from STM32 was verified using a serial terminal on the Raspberry Pi. Data packets were received and parsed correctly during initial testing.

8.3.1.2 Problems Faced

144. UART voltage level mismatch between STM32 (3.3V) and Raspberry Pi (3.3V) required verification to confirm compatibility.
145. Data packet synchronization was inconsistent during initial testing due to missing start and end markers.

8.3.1.3 Solutions Implemented

- 146. Voltage levels were confirmed compatible at 3.3V on both platforms, so no level shifter was required.
- 147. Start and end frame markers were added to the UART data packet format to enable reliable synchronization on the Raspberry Pi receiver.

8.3.2 Day 21 (16/06/2026)

8.3.2.1 Work Done

Python logging script was developed on Raspberry Pi. The script opened the serial port, read incoming UART data packets, parsed the structured data, and stored log entries in a CSV file with timestamps. The logging script was tested over an extended period to verify data continuity and file write reliability.

CSV log file structure was designed to include columns for timestamp, safety state, LIDAR distance, accelerometer X/Y/Z values, and alert type for comprehensive system monitoring.

8.3.2.2 Problems Faced

- 148. CSV file writing occasionally caused brief delays that interrupted serial data reception.
- 149. Log file grew very quickly during continuous high-frequency logging.

8.3.2.3 Solutions Implemented

- 150. Buffered file writing was implemented to batch log entries and reduce write frequency.
- 151. Logging rate was set to one entry per 500 milliseconds to balance data resolution with log file size management.

8.3.3 Day 22 (17/06/2026)

8.3.3.1 Work Done

Dashboard visualization was developed using Python with matplotlib for real-time plotting of LIDAR distance trends, safety state history, and alert event markers. The dashboard updated automatically as new log data arrived from the STM32 ECU.

LCD display content was also updated to show comprehensive system information including current safety state, distance reading, IMU acceleration, and active alert status in a structured multi-line display format.

8.3.3.2 Problems Faced

- 152. Real-time matplotlib plot updates caused high CPU usage on the Raspberry Pi during extended operation.
- 153. LCD display update rate had to be balanced with sensor acquisition and CAN transmission timing.

8.3.3.3 Solutions Implemented

- 154. Dashboard refresh rate was reduced to once per second to reduce CPU load while maintaining useful visualization updates.
- 155. LCD update was scheduled at a fixed interval independent of the main sensor loop to avoid timing conflicts.

8.3.4 Day 23 (18/06/2026)

8.3.4.1 Work Done

Complete communication and logging pipeline testing was performed. The entire system from sensor acquisition to safety state classification to CAN brake command transmission to UART logging to dashboard visualization was run continuously and validated end-to-end.

Log data from extended test runs was analyzed to verify data completeness, state transition accuracy, and alert event logging correctness.

8.3.4.2 Problems Faced

- 156. Extended continuous operation revealed occasional UART data loss events during periods of high CAN bus activity.

8.3.4.3 Solutions Implemented

- 157. UART transmission was given lower interrupt priority than CAN communication to prevent interference, and buffer sizes were increased to handle burst data conditions.

8.3.5 Day 24 (19/06/2026)

8.3.5.1 Work Done

System monitoring and logging were validated through repeated test scenarios including stationary obstacle detection, slow approach, and rapid approach tests. Log files were reviewed to confirm all state transitions and alert events were captured accurately with correct timestamps.

Final preparations for Week 6 comprehensive system testing and documentation were outlined.

8.3.5.2 Problems Faced

158. Dashboard visualization occasionally lost synchronization with live data during rapid state changes.

8.3.5.3 Solutions Implemented

159. Dashboard data queue was implemented to buffer incoming data points and prevent synchronization loss during rapid state changes.

8.4 Work Completed at the End of Week 5

By the end of Week 5, the following tasks were completed successfully:

- 160. UART communication between STM32 and Raspberry Pi established
- 161. Structured data packet format for log transmission designed
- 162. Python logging script developed and tested
- 163. CSV-based data logging with timestamps implemented
- 164. Real-time dashboard visualization developed
- 165. LCD multi-parameter display implemented
- 166. End-to-end communication and logging pipeline validated
- 167. Extended operation stability verified

Week 5 successfully established the data logging and monitoring infrastructure of the project, enabling comprehensive real-time visualization and post-analysis of system performance during safety supervisor ECU operation.



9 TESTING AND DOCUMENTATION (Week 6)

9.1 Introduction

The sixth and final week of the project focused on comprehensive system testing, performance validation, and documentation. During this phase, the complete Safety Supervisor ECU system was tested under a variety of obstacle scenarios to validate detection accuracy, state transition reliability, brake command response time, and overall system performance. Final documentation including architecture diagrams, test results, and the complete project report was prepared.

9.2 Week Plan

The planned activities for Week 6 were:

- 168. Perform complete system integration testing
- 169. Test all safety states with different obstacle scenarios
- 170. Measure and record system response times
- 171. Verify brake assist stopping distances at different robot speeds
- 172. Validate logging and dashboard accuracy against observed system behavior
- 173. Prepare architecture diagrams and system documentation
- 174. Finalize project report
- 175. Prepare demonstration setup

9.3 Day-Wise Work Progress

9.3.1 Day 25 (22/06/2026)

9.3.1.1 Work Done

Comprehensive system integration testing was initiated. All hardware components including STM32 Nucleo, LIDAR, MPU6050, MCP2515 CAN modules, FireBird V Robot, Raspberry Pi, LCD, LEDs, and buzzer were assembled and verified for correct interconnection.

Static obstacle detection testing was performed. Objects were placed at fixed distances within each safety state zone and system responses were verified to be correct and consistent across multiple test repetitions.

9.3.1.2 Problems Faced

- 176. Minor wiring connections became loose during robot movement tests, causing intermittent sensor readings.

9.3.1.3 Solutions Implemented

- 177. All wiring connections were secured and verified before dynamic testing commenced.

9.3.2 Day 26 (23/06/2026)

9.3.2.1 Work Done

Dynamic obstacle testing was performed with the FireBird V Robot in motion. The robot was driven toward stationary obstacles at different speeds to verify that Warning and Emergency states were triggered at correct distances and that brake commands resulted in appropriate speed reduction and motor stop responses.

System response time measurements were taken from obstacle entry into Warning zone to LED alert activation, and from Emergency zone entry to motor stop command reception on the FireBird V. Results were recorded for documentation.

9.3.2.2 Problems Faced

178. Response time varied slightly between test runs due to CAN bus arbitration timing differences.

9.3.2.3 Solutions Implemented

179. Multiple test runs were performed and average response times were calculated for documentation.

9.3.3 Day 27 (24/06/2026)

9.3.3.1 Work Done

Extended reliability testing was performed with the complete system running continuously for an extended period. The system was monitored for stability issues, sensor timeouts, communication errors, and state classification anomalies. Log data was reviewed to confirm consistent system behavior throughout the extended test period.

Architecture diagrams depicting the overall system architecture, communication interfaces, and safety state machine were prepared for the project report.

9.3.3.2 Problems Faced

180. Extended testing revealed that Raspberry Pi CPU temperature increased during long-duration logging and visualization.

9.3.3.3 Solutions Implemented

181. Dashboard refresh rate was further reduced and an external USB fan was used to manage Raspberry Pi temperature during extended testing.

9.3.4 Day 28 & Day 29 (25/06/2026 – 26/06/2026)

9.3.4.1 Work Done

Final project report documentation was prepared. All s including introduction, literature review, hardware and software descriptions, weekly work progress, test results, future improvements, and conclusion were compiled and formatted. Final system performance summary was documented based on testing results.

Demonstration setup was prepared and a final end-to-end system demonstration was conducted to verify complete functionality before report submission. The demonstration successfully showed real-time obstacle detection, safety state classification, LED and buzzer alerts, CAN-based brake command transmission, motor speed reduction, emergency stop, and Raspberry Pi data logging.

9.3.4.2 Problems Faced

182. Minor formatting inconsistencies were found in the initial report draft.

9.3.4.3 Solutions Implemented

183. Report was reviewed and formatting was corrected for consistency throughout all s.

9.4 Work Completed at the End of Week 6

By the end of Week 6, the following tasks were completed successfully:

184. Complete system integration testing performed
185. Static and dynamic obstacle detection testing completed
186. System response time measurements recorded
187. Brake assist stopping distance validation completed
188. Extended reliability testing completed
189. Architecture diagrams and system documentation prepared
190. Final project report compiled and formatted
191. Complete system demonstration conducted successfully

Week 6 successfully validated the complete Safety Supervisor ECU system performance and completed all project documentation, demonstrating a functional prototype of an automotive emergency brake assist system using embedded AI, CAN communication, and real-time sensor processing.

FUTURE IMPROVEMENTS

The developed Safety Supervisor ECU prototype can be significantly improved and extended through several future development directions.

GPS-based vehicle tracking and route monitoring can be integrated to enable fleet management and remote supervision of vehicle locations and movement patterns. This would allow school transportation operators or fleet managers to monitor all vehicles in real time from a central control system.

Cloud-based data logging systems can be implemented to store sensor data, safety state histories, alert events, and CAN communication logs remotely. Internet connectivity through Wi-Fi or cellular modules would enable real-time data access, historical analysis, and remote monitoring capabilities for the Safety Supervisor ECU.

Multi-sensor fusion using camera, radar, and LIDAR inputs simultaneously can significantly improve obstacle detection reliability and reduce false positive alerts. Deep learning-based object classification can be added to distinguish between different obstacle types such as vehicles, pedestrians, and stationary objects for more intelligent safety responses.

Driver drowsiness detection using facial monitoring cameras and biometric sensors can be integrated into the same hardware platform to add another layer of driver assistance functionality. Integration with vehicle control systems through direct CAN bus access would enable fully automatic speed control rather than prototype motor simulation.

Advanced embedded AI processing using dedicated neural processing hardware can be integrated to perform real-time object detection, scene understanding, and predictive collision avoidance directly on the ECU without requiring external processing units.

FreeRTOS real-time operating system integration can improve task scheduling, timing determinism, and system reliability for safety-critical ECU applications where timing guarantees are essential.

Real Vehicle Scenario

No .	Real Vehicle Scenario	Risk Level	ECU Solution / Safety Response
1	Empty road, no vehicle or obstacle ahead.	Low	Remain in Normal state. Continue monitoring distance, IMU motion, and CAN health. No brake command is issued.
2	Lead vehicle is present but maintaining a safe distance.	Low	Keep the vehicle in Normal state. Maintain normal driving and update the dashboard with safe following distance information.
3	Lead vehicle gradually slows down in the same lane.	Medium	Enter Warning state if the distance reduces below the warning threshold. Alert the driver and prepare for speed reduction.
4	Lead vehicle suddenly applies hard brakes.	High	Enter Emergency state if the time-to-collision becomes unsafe. Cut drive command and immediately apply emergency braking.
5	Pedestrian starts crossing in front of the vehicle.	High	If the pedestrian enters the unsafe distance, command emergency braking immediately. Log the event as a pedestrian crossing risk.
6	Child suddenly runs from behind a parked vehicle or near a school zone.	Very High	Give the highest safety priority. Activate immediate emergency braking, sound the buzzer, and display a black-box event capture.
7	Two-wheeler suddenly cuts into the lane.	High	Continuously monitor the rapidly changing distance. Enter Warning for moderate risk and Emergency braking for collision-level risk.
8	Vehicle ahead changes lane and reveals a stopped vehicle in front.	High	Detect the newly visible obstacle and quickly transition from Normal to Warning or Emergency depending on the remaining distance.

9	Traffic signal ahead has a queue of stopped vehicles.	High	Reduce speed early in the Warning state. Apply Emergency braking if the stopping distance becomes critical.
10	Stop-and-go city traffic with repeated close distances.	Medium	Use a controlled Warning state and filtering to avoid unnecessary braking. Maintain a safe following distance.
11	Animal suddenly appears on the road.	High	Treat the animal as a dynamic obstacle. Apply Warning or Emergency braking depending on obstacle distance and vehicle speed.
12	Construction barrier or road blockage appears ahead.	High	Detect the fixed obstacle, reduce vehicle speed, and stop if necessary. Keep the brake command active until the obstacle is cleared.
13	Vehicle is taking a turn and the sensor detects a side object or divider.	Medium	Avoid false emergency braking using path relevance logic. Apply braking only if the object is located in the vehicle's driving path.
14	Speed breaker or pothole causes sudden vibration or tilt.	Medium	Use IMU data to identify abnormal motion. Prevent obstacle braking caused only by vibration and continue monitoring road conditions.
15	Rain, fog, dust, or reflections create noisy distance readings.	Medium	Use repeated sensor sampling and validity checks. If confidence is low, enter a degraded safe mode and alert the driver.

CONCLUSION

The "Safety Supervisor ECU with Emergency Brake Assist" project successfully demonstrated the implementation of a prototype automotive safety system using STM32 Nucleo-F446RE, LIDAR Lite v3HP, MPU6050 IMU, MCP2515 CAN modules, FireBird V Robot, and Raspberry Pi.

The project addressed the need for affordable intelligent vehicle safety systems by developing a prototype Safety Supervisor ECU capable of real-time obstacle detection, three-level safety state classification, multi-modal driver alert generation, and automatic emergency brake control through CAN communication. The integration of embedded systems, sensor fusion, real-time state machine logic, CAN communication, and data logging created a functional prototype suitable for demonstrating automotive ADAS principles.

The project provided practical understanding regarding embedded ECU design, I2C sensor interfacing, CAN bus communication implementation, real-time safety logic development, motor control integration, UART-based data logging, and Raspberry Pi visualization — all fundamental skills for automotive embedded systems engineering.

Despite challenges such as sensor communication debugging, CAN bus configuration, motor control integration, and system timing optimization, the project successfully achieved end-to-end Safety Supervisor ECU functionality, establishing a strong foundation for future automotive ADAS development.

With further optimization and integration of additional safety features such as multi-sensor fusion, GPS tracking, cloud logging, and direct vehicle ECU integration, the system has the potential to evolve into a comprehensive automotive safety platform capable of significantly improving road safety and reducing accident risks.

REFERENCES

- [1] STMicroelectronics, "UM1724 – STM32 Nucleo-64 Boards (MB1136) User Manual," 2025.
- [2] Garmin, "LIDAR-Lite v3HP Operation Manual and Technical Specifications," 2023.
- [3] InvenSense, "MPU-6000 and MPU-6050 Product Specification Revision 3.4," 2013.
- [4] Microchip Technology, "MCP2515 Stand-Alone CAN Controller with SPI Interface Datasheet," 2019.
- [5] R. Rajamani, Vehicle Dynamics and Control, 2nd ed. New York: Springer, 2012.
- [6] M. Maurer, J. C. Gerdes, B. Lenz, and H. Winner, Autonomous Driving: Technical, Legal and Social Aspects. Berlin: Springer, 2016.
- [7] Raspberry Pi Foundation, "Raspberry Pi Hardware Documentation," 2024. [Online]. Available: <https://www.raspberrypi.com/documentation/>
- [8] Embedded Systems Academy, "CAN Protocol Implementation Guide for Automotive Applications," 2021.

ACKNOWLEDGEMENT

The successful completion of this internship project would not have been possible without the guidance, support, and encouragement received from various individuals and organizations throughout the development process.

First and foremost, sincere gratitude is expressed to KPIT APEX Lab for providing the opportunity to work on the project titled "Safety Supervisor ECU with Emergency Brake Assist" as part of the Summer Internship Program. The internship provided valuable exposure to real-world automotive embedded systems, CAN communication protocols, sensor integration, and Advanced Driver Assistance Systems (ADAS) development.

Special thanks are extended to our mentor, Isheet Patel Sir, for providing the project problem statement, continuous technical guidance, valuable suggestions, and regular project review meetings conducted throughout the internship duration. His mentorship played an important role in understanding project requirements, solving technical challenges, and improving the implementation approach during each phase of development.

Gratitude is also expressed to Parag Narkhede Sir, Suraj Vashishtha sir and colleagues who provided support during hardware setup, software debugging, testing, documentation, and problem-solving activities throughout the internship period.

Special appreciation is also extended to the developers and contributors of open-source technologies such as Raspberry Pi OS, Python libraries, and Arduino IDE, which made implementation of this project possible using affordable embedded hardware platforms.